



南開大學
Nankai University

计算机学院

并行程序设计实验报告

MPI 编程实验

ANN 近似最近邻搜索

多进程数据划分、进程内 OpenMP 与图索引组合

IVF-PQ / Block-HNSW / IVF-HNSW / HNSW-on-HNSW

姓名：柯云超

学号：2413575

专业：计算机科学与技术

2026 年 5 月 25 日

目录

摘要	2
1 问题定义与实验目标	2
2 系统结构	3
3 算法与并行实现	5
4 实验环境与复现方法	6
5 参数扫描与 Recall-Latency 权衡	8
6 强扩展性分析	11
7 通信开销与负载均衡	14
8 VTune 性能剖析	16
9 阻塞与非阻塞通信实验	18
10 结果可信性与可复现性检查	19
11 问题与改进方向	20
12 结论	20
A 复现命令与结果文件	21
B 实验矩阵明细	22

摘要

本实验在前两次 ANN 实验已有的 SIMD 距离内核、IVF-PQ 和 HNSW 结构基础上，将搜索任务迁移到 MPI 多进程环境，并在每个 MPI rank 内继续使用 OpenMP 做进程内并行。程序采用统一入口 `main.cc`，支持四种搜索模式：IVF-PQ、分块 HNSW、IVF+HNSW 嵌套索引，以及 HNSW-on-HNSW 分层图索引。MPI 层负责 base 向量划分、query 广播、候选集 gather 和 rank 0 的 top- k 合并；OpenMP 层负责本地倒排表、HNSW 块或候选块之间的并行搜索。

实验使用 DEEP100K 数据集，距离度量为内积距离，统一评价 Recall@10、平均单 query 延迟、本地最慢 rank 搜索时间、通信与合并开销，以及各 rank 搜索时间分布。正式结果只展示 Windows MS-MPI 和鲲鹏 PBS 两个平台。参数扫描覆盖 IVF-PQ、Block-HNSW、IVF+HNSW 和 HNSW-on-HNSW 的核心参数；扩展性实验覆盖 $np=1,2,4,8$ ；通信模式实验在鲲鹏 PBS 上比较阻塞 gather 与非阻塞 gather。结果表明：相同参数下两平台 Recall 完全一致；在 $np=8$ ， $threads=2$ ， $query=2000$ 的代表配置中，通信与 merge 占比通常低于 1.1%，主要瓶颈仍在各 rank 的本地 ANN 搜索。图索引方面，Block-HNSW 达到 Recall@10=1.00000；HNSW-on-HNSW 在全 block 探测配置下达到 Recall@10=0.99995，但延迟显著更高，体现了典型 recall-latency trade-off。

关键词： ANN；MPI；OpenMP；IVF-PQ；HNSW；HNSW-on-HNSW；负载均衡；通信开销

1 问题定义与实验目标

给定基向量集合

$$X = \{x_0, x_1, \dots, x_{N-1}\}, \quad x_i \in \mathbb{R}^{96},$$

以及 query 向量 q ，ANN 搜索返回与 q 最接近的 k 个候选。本实验沿用前两次 ANN 作业的内积距离：

$$d(x, q) = 1 - \sum_{j=0}^{d-1} x_j q_j.$$

Recall@ k 定义为返回集合与 ground truth top- k 的交集比例：

$$\text{Recall@}k = \frac{|KNN(q) \cap KANN(q)|}{k}.$$

与 Pthread/OpenMP 实验不同，本次实验关注多进程之间如何划分数据、如何通信、如何将每个 rank 的局部结果归并为全局 top- k 。因此报告重点围绕以下问题展开：

1. base data 如何划分到不同 MPI 进程，且尽量保持负载均衡；
2. query 如何广播，各 rank 如何维护本地索引并返回局部 top- k ；
3. rank 0 的 gather/merge 开销相对本地搜索是否显著；
4. MPI 与 OpenMP 如何组合，进程间并行和进程内并行的边界在哪里；
5. IVF-PQ 与三类图索引方案在 recall-latency 上有什么差异；
6. Windows MS-MPI 与鲲鹏 PBS 两个平台上是否能得到一致且可复现的结果。

代码与脚本均放在 `ann-mpi/` 目录下。正式提交入口为 `main.cc` 与 `qsub_mpi.sh`；报告中引用的运行记录分别来自 Windows 本地全量日志与鲲鹏 PBS 全量日志，均保存于 `results/` 目录。

关注点	本报告回答的问题	对应章节
MPI 数据流	base 如何切分、query 如何广播、候选如何合并。	第 2、3 节
参数 trade-off	不同 <code>nprobe</code> 、 <code>ef</code> 、 <code>nlist</code> 与召回/延迟的关系。	第 5 节
扩展性	$np=1,2,4,8$ 下的强扩展、效率和通信变化。	第 6 节
混合布局	固定 worker 预算下，MPI 进程数与 rank 内线程数如何分配。	第 6 节
负载均衡	各 rank 搜索时间是否接近，最慢 rank 是否拖尾。	第 7 节
性能剖析	VTune 符号化热点、汇编观察与硬件事件实测。	第 8 节
通信模式	阻塞 MPI 与非阻塞 MPI 的实验对比。	第 9 节

表 1 正文阅读线索

2 系统结构

2.1 MPI 数据流

整体结构如图 1 所示。rank 0 负责读取 DEEP100K 的 base/query/ground truth 文件，然后通过 `MPI_Scatterv` 将 base 向量按连续区间分给所有 rank。每个 rank 只在自己的 `local_base` 上构建本地索引。在线查询阶段，rank 0 将 query 批量 `MPI_Bcast` 到所有进程；每个进程独立搜索本地 shard，打包本地 top- k 候选，再通过 gather 汇总到 rank 0；最后 rank 0 对所有候选做全局 top- k merge，并计算 Recall@10。

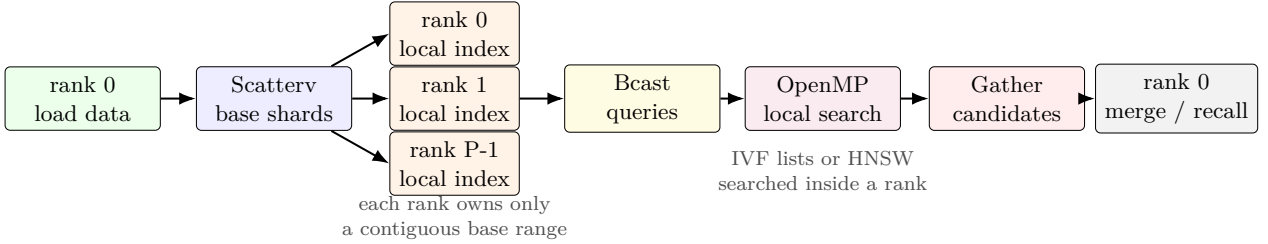


图 1 MPI + OpenMP 混合 ANN 搜索流程。MPI 负责跨进程数据分发和结果归并，OpenMP 只作用于每个 rank 的本地搜索。

2.2 负载划分

代码中使用 `PartitionRange(n, rank, world_size)` 对 base 向量做连续分块。设进程数为 P ，则每个 rank 得到

$$\left\lfloor \frac{N}{P} \right\rfloor \quad \text{或} \quad \left\lceil \frac{N}{P} \right\rceil$$

个向量，最大只差 1。连续划分的优点是实现简单、传输时可直接使用 `MPI_Scatterv` 的 displacement，同时每个 rank 的 `local_base` 连续存储，有利于本地 cache locality。缺点是如果数据分布本身有明显偏斜，某些 rank 的搜索路径可能更长；因此程序额外打印 `per-rank search latency (us)`，并用

$$\text{imbalance} = \frac{\max_i T_i}{\frac{1}{P} \sum_i T_i}$$

衡量最慢 rank 与平均 rank 的差异。

```

1 Range PartitionRange(size_t n, int rank, int world_size) {
2     const size_t base = n / world_size;
3     const size_t extra = n % world_size;
4     const size_t begin = rank * base + std::min<size_t>(rank, extra);
5     const size_t count = base + (rank < extra ? 1 : 0);
6     return {begin, begin + count};
7 }
  
```

2.3 局部搜索与全局合并

每个 rank 的局部搜索结果以小顶堆形式保存。为了跨进程传输，程序将每个 query 的 top- k 打包成两个定长数组：距离 float 和全局 id uint64_t。rank 0 收到所有 rank 的候选后，对每个 query 扫描 $P \times k$ 个候选并维护全局 top- k 堆。因此 merge 的复杂度近似为

$$O(QPk \log k),$$

其中 Q 是 query 数。与本地搜索的 IVF 或 HNSW 访问相比，当 P 和 k 不大时，这部分开销很小。正式全量实验中 $P=8, k=10, Q=2000$ ，每个 rank 发送约 $Qk(4+8) \approx 234$ KiB 候选数据，query 广播约 $Qd \cdot 4 \approx 750$ KiB，因此通信量仍小于本地索引搜索的访存量。

候选缓冲区按“query 优先、rank 次之、候选序号最后”的顺序组织。这样做的原因是 rank 0 在合并时天然按 query 扫描，内层只需要从每个 rank 取出该 query 的 k 个局部候选，不必重新解析变长消息。图 2 给出了这一布局 and 合并过程。

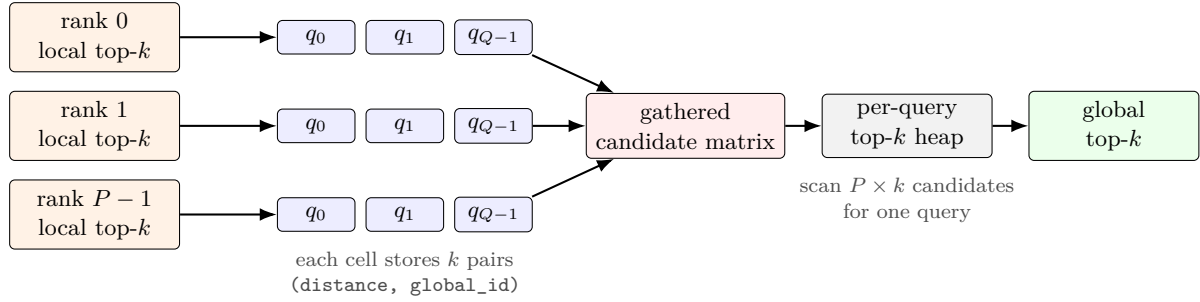


图 2 rank 0 的候选矩阵与 top- k 合并过程。每个 rank 只发送本地 top- k ，最终输出仍按全局 id 计算 Recall。

合并逻辑对应的伪代码如下。代码中距离越小代表越接近，因此堆顶保存当前全局 top- k 中最差的候选；新候选更好时替换堆顶。

```

1 for q in 0 .. query_n-1:
2     heap = empty max_heap_by_distance()
3     for r in 0 .. world_size-1:
4         for j in 0 .. k-1:
5             cand = gathered[r][q][j]
6             if cand.id is invalid:
7                 continue
8             if heap.size() < k:
9                 heap.push(cand)
10            else if cand.distance < heap.top().distance:
11                heap.pop()
12                heap.push(cand)
13 output[q] = sort_by_distance(heap)

```

2.4 代码组织与数据路径

本目录由上一阶段 ANN 多线程实验扩展而来，继续复用 SIMD、IVF、HNSW 内核，同时清理掉与 MPI 提交无关的旧脚本、临时输出和构建产物。最终报告对应的主要文件见表 2。这样组织的好处是：提交入口足够清楚，实验结果和报告互相引用，服务器上只需要重新编译 main.cc 并走 PBS 脚本即可复现实验。

文件或目录	作用
main.cc	MPI + OpenMP 统一入口，完成参数解析、数据分发、本地索引构建、搜索、gather 和 rank 0 merge。
ivf/	复用 IVF-PQ 索引和 OpenMP 搜索实现，作为本次 IVF 基础迁移方案。
hns/	保存 Block-HNSW、IVF+HNSW 和 HNSW-on-HNSW 相关实现。
qsub_mpi.sh	按提交指南使用 PBS 启动作业，支持用环境变量覆盖进程数、线程数和算法参数。
scripts/	保存 Windows 本地实验、鲲鹏 PBS 自动提交、结果解析和绘图脚本。
results/	保存原始日志、解析 CSV 与 VTune 导出，所有图表数字都可回溯到这些文件。
report/	保存本报告源码、校徽资源、生成图和最终 PDF。

表 2 ANN MPI 实验目录中的关键文件

数据路径仍沿用前两次实验的约定：程序优先读取 ANN_DATA_PATH，服务器上设置为 /anndata，本地 Windows 脚本设置为仓库外层的 files/ 数据目录。如果环境变量未设置，程序会按平台尝试默认路径。这样做避免把 DEEP100K 数据复制进提交目录，也使 Windows 与鲲鹏 PBS 两边的命令只在数据路径和作业调度上不同。

3 算法与并行实现

3.1 四种搜索模式

本次实现不只保留一个 baseline，而是把 IVF 与图索引分支放进同一套 MPI 外壳中。四种模式的命令行接口如表 3 所示。

模式	运行形式	实现要点
IVF-PQ	<code>./main t nlist nprobe p q local</code>	每个 rank 在本地 shard 上构建 IVF-PQ，OpenMP 按 query 并行搜索倒排表，再对 top- p 候选 rerank。
Block-HNSW	<code>./main t m ef p q hnsw</code>	base 先由 MPI 切为多个 shard，每个 shard 单独建立 HNSW；rank 内使用多入口 OpenMP 搜索，rank 0 合并。
IVF+HNSW	<code>./main t nlist nprobe ef q ivf-hnsw</code>	每个 rank 内先按 IVF 聚类，再在每个局部簇内建立 HNSW；搜索时先选簇，再搜索簇内图。
HNSW-on-HNSW	<code>./main t nblocks nprobe ef q hnsw-on-hnsw</code>	每个 rank 把本地 shard 分块，先在块质心上建立 top-level HNSW，再为每个块建立 bottom-level HNSW。

表 3 统一 MPI 入口下的四种 ANN 搜索模式

3.2 IVF-PQ 的 MPI 迁移

IVF-PQ 是从上一阶段多线程版本迁移到 MPI 的基础方案。多线程版本中，一个进程维护完整索引；MPI 版本中，每个 rank 只维护本地 shard 的 inverted lists。query 被广播到所有进程后，各 rank 只在本地 list 中搜索，返回局部 top- k 。由于每个 rank 只看到一部分 base data，最终的 top- k 必须由 rank 0 对所有局部结果进行全局合并。

进程内仍复用 OpenMP inter-query 并行：当 query 批量足够大时，不同线程处理不同 query，写入不同结果槽，几乎不需要同步。这一点延续了前一次 Pthread/OpenMP 实验中“inter-query 粒度更稳健”的结论。

3.3 图索引 A/B/C

图索引部分实现了三种结构，分别对应“先筛选再图搜索”“分 rank 图搜索”和“分层图搜索”，结构如图 3。

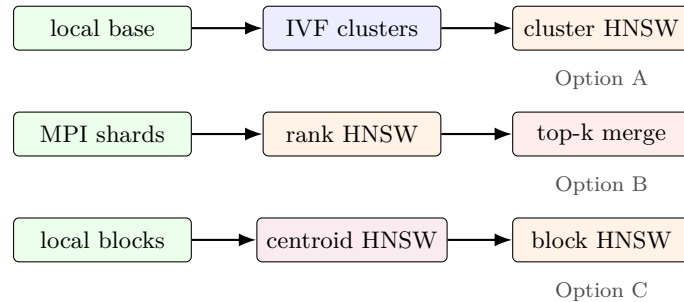


图 3 三类图索引方案：IVF+HNSW、分块 HNSW 和 HNSW-on-HNSW。

方案 A：IVF+HNSW。每个 rank 内先建立 IVF 粗聚类，随后在每个簇中单独建立 HNSW。搜索阶段先按 query 与质心距离选择 $nprobe$ 个簇，再在这些簇的 HNSW 内搜索。它的优点是能避免搜索所有局部向量；缺点是召回依赖簇选择，若 ground truth 所在簇未被探测，则后续 HNSW 再精确也无法恢复。

方案 B：分块 HNSW。MPI 本身已经把 base 划分成多个 shard，因此每个 rank 的 HNSW 可以视为一个独立 block graph。所有 rank 并行搜索自己的图，rank 0 再合并结果。该方案实现直接、召回较高，但每个局部 HNSW 的搜索路径具有串行依赖，OpenMP 只能在多入口或多 query 层面分摊，不能把单条贪心路径线性拆开。

方案 C: HNSW-on-HNSW。本实验进一步实现了分层图：每个 rank 把本地 base 分成多个 block，先计算 block centroid 并建立 top-level HNSW；每个 block 内再建立 bottom-level HNSW。搜索时先在 centroid graph 上选择若干 block，再并行搜索被选中的 block graph。它能表达“图上再建图”的组合策略：当 `nprobe_blocks` 取满 16 时，全量实验 Recall@10 达到 0.99995，但因为实际搜索了所有 block，延迟明显上升。

3.4 混合并行边界

本实验的混合并行不是简单地同时打开 MPI 和 OpenMP，而是明确划分职责：

- MPI 层：base shard 分配、query 广播、候选 gather、最慢 rank 计时归约、rank 0 top- k merge；
- OpenMP 层：rank 内 query/list/block 级并行，避免跨进程共享状态；
- 索引层：每个 rank 独立构建本地 IVF-PQ 或 HNSW，不在在线阶段跨进程访问远端索引。

这种边界使程序比较容易在 Windows MS-MPI 和鲲鹏 PBS 环境中复现。它也限制了通信复杂度：在线阶段只有一次 query broadcast 和一次候选 gather，没有在 HNSW 搜索路径中进行细粒度跨 rank 通信。

3.5 在线阶段伪代码

为了说明 MPI 代码的控制流，下面给出一次批量查询的伪代码。索引构建发生在计时前；报告中的 online latency 从 query broadcast 前的 barrier 之后开始计时，直到 rank 0 完成 merge 和 recall 计算前结束。

```

1 range = PartitionRange(base_n, rank, world_size);
2 MPI_Scatterv(root_base, counts, displs, MPI_FLOAT, local_base);
3 index = BuildLocalIndex(local_base, mode);
4
5 MPI_Barrier(MPI_COMM_WORLD);
6 phase_begin = MPI_Wtime();
7 MPI_Bcast(queries, query_n * dim, MPI_FLOAT, 0, MPI_COMM_WORLD);
8
9 search_begin = MPI_Wtime();
10 local_topk = SearchLocal(index, queries, nthreads);
11 search_us = MPI_Wtime() - search_begin;
12
13 PackCandidates(local_topk, range.begin, local_distances, local_ids);
14 GatherCandidates(local_distances, local_ids, all_distances, all_ids);
15 MPI_Reduce(search_us, max_search_us, MPI_MAX, 0, MPI_COMM_WORLD);
16
17 if (rank == 0) {
18     merged = MergeGatheredCandidates(all_distances, all_ids);
19     total_us = MPI_Wtime() - phase_begin;
20     comm_merge_us = total_us - max_search_us;
21     recall = RecallAtK(merged, ground_truth);
22 }
```

这里使用 `MPI_Reduce` 的 `MPI_MAX` 而不是平均值，是因为一次分布式查询必须等最慢 rank 完成本地搜索后才能合并。因此总延迟应与最慢 rank 的搜索时间对齐，`comm+merge latency` 才能表示“除了最慢本地搜索以外的剩余开销”。如果用平均搜索时间，会把负载不均衡误计入通信开销。

4 实验环境与复现方法

4.1 平台与数据集

实验使用 DEEP100K 数据集：base 向量 $N = 100000$ ，维度 $d = 96$ ，query 和 ground truth 使用课程提供的二进制文件。正式实验统一取前 `query_n=2000` 条 query， $k = 10$ 。平台信息见表 4。

平台	MPI/编译环境	用途
Windows 本地	MS-MPI, MinGW-w64, AVX2/FMA 参数	本地开发、真实 MPI 参数扫描、扩展性、混合布局、亲和性实验，以及 rank-local VTune 剖析。
鲲鹏 PBS	AArch64 Linux, 服务器 MPI 编译器, PBS 队列, /anndata 共享数据	按提交指南验证作业提交路径，完成同样的参数扫描、扩展性实验和通信模式实验。

表 4 实验平台与用途。报告正文只展示 Windows MS-MPI 与鲲鹏 PBS 两个平台结果。

4.2 构建与运行脚本

Windows 本地全量参数扫描、扩展性和进阶实验使用 PowerShell 脚本：

```
1 powershell -ExecutionPolicy Bypass -File scripts/run_parameter_sweep.ps1
2 powershell -ExecutionPolicy Bypass -File scripts/run_scalability.ps1
3 powershell -ExecutionPolicy Bypass -File scripts/run_hybrid_layout.ps1
4 powershell -ExecutionPolicy Bypass -File scripts/run_affinity.ps1
```

鲲鹏 PBS 实验由自动脚本通过跳板机提交作业，并在作业完成后拉回 `test.o/test.e` 摘要：

```
1 micromamba run -n test python scripts/run_kunpeng_pbs_sweep_auto.py
2 micromamba run -n test python scripts/run_kunpeng_pbs_comm_auto.py
```

绘图与 CSV 解析固定使用用户指定的本地环境：

```
1 micromamba run -n test python scripts/parse_mpi_results.py
```

4.3 结果字段与计时口径

程序为所有模式输出统一字段：

- `average recall`: Recall@10;
- `average latency (us)`: 从 query broadcast 到 rank 0 merge 完成的平均单 query 时间;
- `max local search latency (us)`: 所有 rank 中最慢本地搜索时间;
- `comm+merge latency (us)`: 总在线时间减去最慢本地搜索时间，近似表示广播、gather、reduce 和 merge 的合计开销;
- `per-rank search latency (us)`: 每个 rank 的本地搜索时间，用于计算负载不均衡度。

4.4 结果可复现性

本次报告没有只摘录屏幕上的最后几行，而是把运行路径固定在脚本中，并保存完整日志。表 5 列出报告数据来源。若重新实验，只需重新运行脚本并执行解析脚本，图和 CSV 即可更新。

文件	平台 / 实验	记录内容
parameter_sweep 20260525_141037.txt	Windows 参数扫描	四种算法的参数 sweep 原始输出。
scalability 20260525_145526.txt	Windows 扩展性	np=1,2,4,8 下四算法延迟、通信和 per-rank 时间。
hybrid_layout 20260525_205224.txt	Windows 混合布局	固定 worker 预算下的 np × threads 布局对比。
affinity 20260525_205832.txt	Windows 亲和性	默认、P-core、E-core 与 P/E 全开的真实 MPI 作业对比。
kunpeng_pbs_sweep 20260525_153105.txt	鲲鹏 PBS 参数与扩展性	24 个参数点和 16 个扩展性点，共 40 个结果。
kunpeng_pbs_comm_modes 20260525_154940.txt	鲲鹏 PBS 通信模式	阻塞与非阻塞 MPI 的四算法对比。
vtune_exports/ vtune_notes.md	与 Windows profiling	VTune 符号化 hotspots、汇编摘录、uarch 普通会话日志与管理员实测导出。

表 5 报告结果对应的可复现日志与导出文件

5 参数扫描与 Recall-Latency 权衡

5.1 代表性参数点

表 6 汇总了两平台在相同代表参数下的结果。所有点均来自参数扫描日志，统一使用 np=8、threads=2、query_n=2000。代表参数如下：

- IVF-PQ: nlist=16、nprobe=4、p=1000;
- Block-HNSW: ef=50;
- IVF+HNSW: nprobe=4、ef=50;
- HNSW-on-HNSW: nblocks=16、nprobe_blocks=16、ef=50。

平台	算法	Recall	latency	local max	comm	imbalance
Windows MS-MPI	IVF-PQ	0.96515	334.59555	331.10550	3.49005	1.06707
Windows MS-MPI	Block-HNSW	1.00000	221.45235	218.78595	2.66640	1.00663
Windows MS-MPI	IVF+HNSW	0.96450	326.40570	322.79650	3.60920	1.00457
Windows MS-MPI	HNSW-on-HNSW	0.99995	2053.35915	2045.68515	7.67400	1.01303
鲲鹏 PBS	IVF-PQ	0.96515	512.09760	509.37176	2.72584	1.04614
鲲鹏 PBS	Block-HNSW	1.00000	765.25044	762.63249	2.61796	1.19441
鲲鹏 PBS	IVF+HNSW	0.96450	911.40282	908.41758	2.98524	1.03207
鲲鹏 PBS	HNSW-on-HNSW	0.99995	2489.16447	2486.21905	2.94542	1.17062

表 6 双平台代表参数点。所有延迟单位均为 $\mu\text{s}/\text{query}$ 。

相同算法参数下，两平台 Recall 完全一致，说明 base shard 的全局 id 偏移、候选 gather 顺序和 rank 0 top- k merge 在跨平台上是稳定的。延迟上，Windows 的图索引明显更快；鲲鹏 PBS 的通信/merge 绝对值更低，但本地 HNSW 图搜索耗时更高，说明主要差异来自图访问、优先队列和分支路径，而不是 MPI 传输。

5.2 Windows 参数曲线

图 4 给出 Windows MS-MPI 上的参数曲线。IVF-PQ 和 IVF+HNSW 的横轴随 nprobe 增大整体右移，同时 Recall 上升；Block-HNSW 随 ef 增大召回更稳定，但图搜索扩展邻居更多；HNSW-on-HNSW 的 nprobe_blocks 越大，越接近搜索全部 block，召回接近 1.0 时延迟迅速增加。

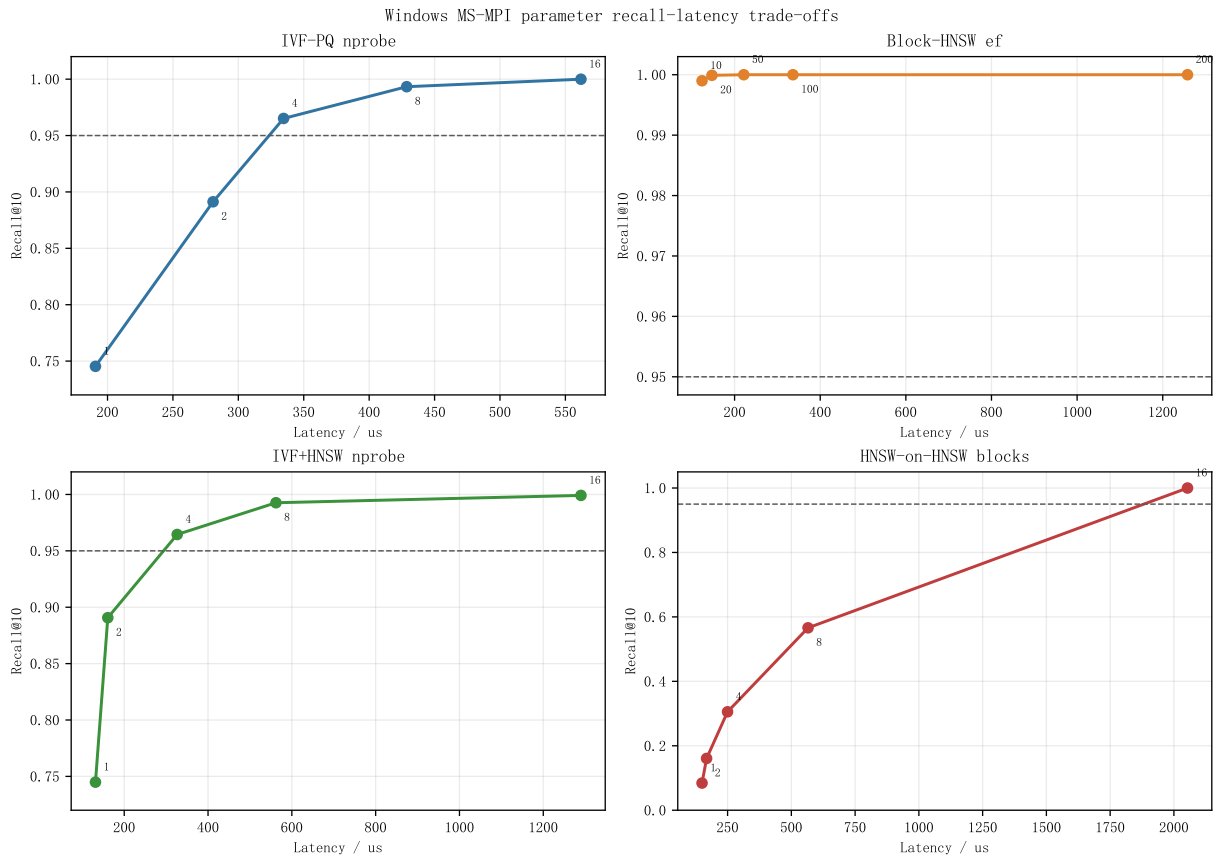


图 4 Windows MS-MPI 参数扫描的 Recall-Latency 曲线。每个子图对应一种算法的核心参数。

5.3 双平台参数曲线

图 5 将同一组参数曲线放到 Windows 与鲲鹏 PBS 两个平台比较。两平台曲线在纵轴上几乎重合，说明参数对召回的影响由算法结构主导；横轴位置不同，说明硬件、MPI runtime 和服务器队列环境决定了绝对延迟。这个结果也解释了为什么报告必须同时给出 recall 和 latency：只看召回无法区分不同平台的性能代价，只看延迟又可能忽略算法返回质量。

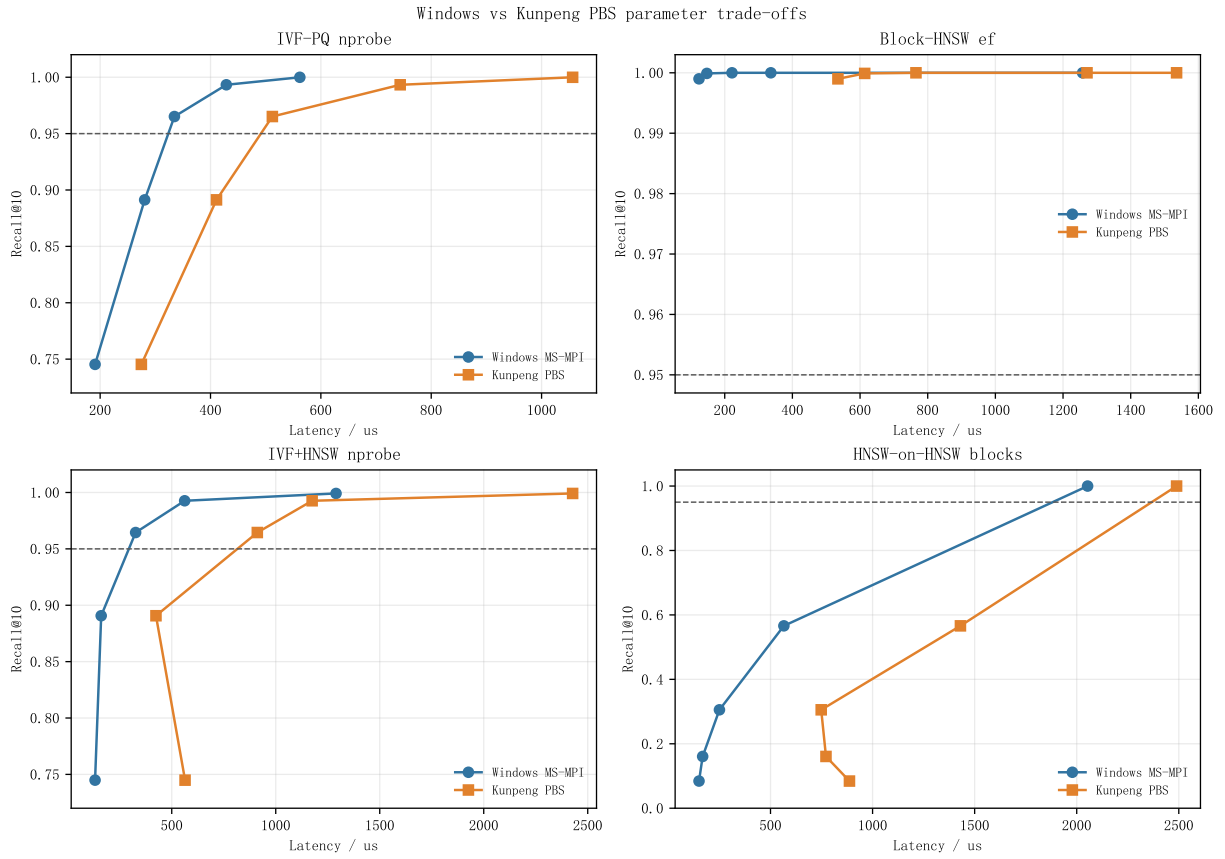
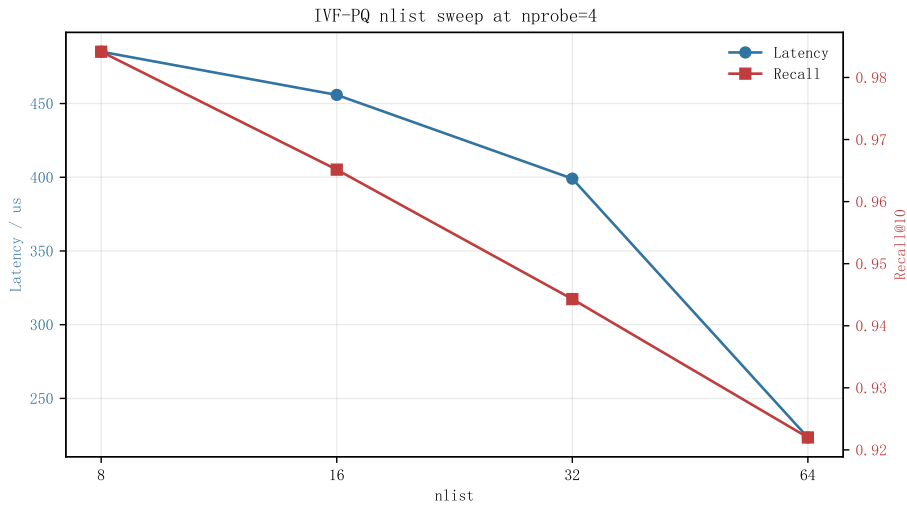


图 5 Windows MS-MPI 与鲲鹏 PBS 的参数 trade-off 对比。

5.4 IVF-PQ 的 nlist 影响

IVF-PQ 额外扫描了 $nlist=8, 16, 32, 64$ 。图 6 固定 $nprobe=4$ ，观察粗聚类数量变化。较小 $nlist$ 会让每个 inverted list 更长，单次探测扫描更多向量；较大 $nlist$ 会降低每个 list 长度，但也可能使同样 $nprobe$ 覆盖的全局空间变窄。实验中 $nlist=16, nprobe=4$ 是召回和延迟较均衡的代表配置。

图 6 IVF-PQ 在 $nprobe=4$ 下的 $nlist$ sweep。

5.5 参数选择建议

从参数扫描可以得到四条较直接的建议。第一，IVF-PQ 若追求 Recall@10 约 0.96， $nlist=16$ 、 $nprobe=4$ 、 $p=1000$ 已经达到较好折中；若继续增加 $nprobe$ ，召回提升变小但延迟增加。第二，Block-

HNSW 在本数据集上最适合做高召回方案， $ef=50$ 已达到 $Recall@10=1.00000$ 。第三，IVF+HNSW 的召回受粗聚类筛选影响， $nprobe$ 太小时即使簇内 HNSW 足够精确也无法恢复漏掉的真实近邻。第四，HNSW-on-HNSW 适合作为分层图结构验证；如果作为实用低延迟方案，应减少 $nprobe_blocks$ 并接受一定 recall trade-off。

6 强扩展性分析

6.1 Windows 强扩展

图 7 给出 Windows 上 $np=1,2,4,8$ 的加速比和并行效率。IVF-PQ 在 4 个进程时达到 $1.30\times$ ，但 8 个进程回落到 $0.99\times$ ；IVF+HNSW 在 2 个进程时达到 $1.32\times$ ，之后因为本地图搜索变短而通信、调度和索引常数项占比上升。Block-HNSW 和 HNSW-on-HNSW 在 Windows 上没有线性加速，说明图搜索的局部结构、内存访问和进程启动/通信成本已经抵消了 shard 变小带来的收益。

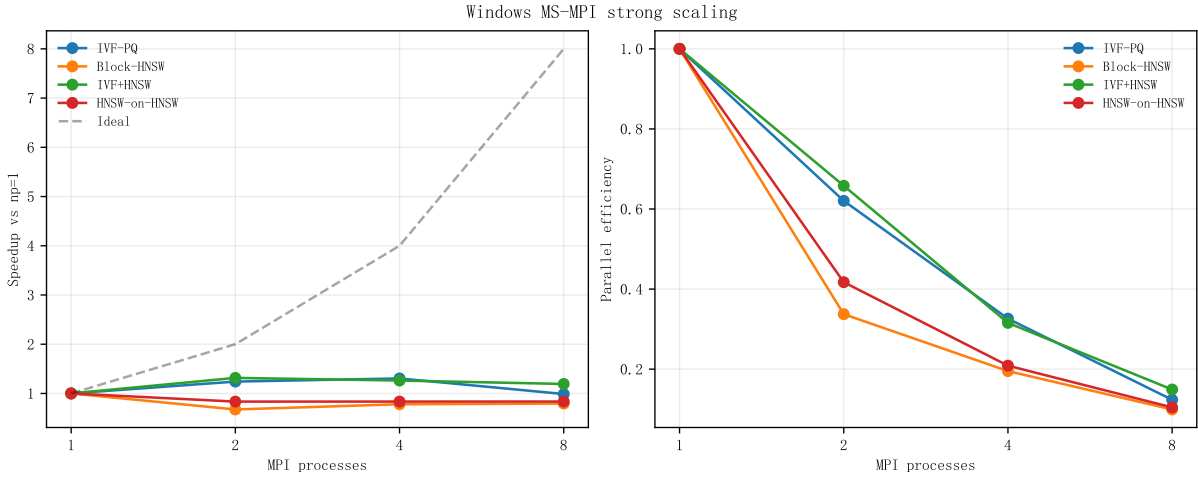


图 7 Windows MS-MPI 强扩展：相对 $np=1$ 的加速比与并行效率。

6.2 双平台扩展性

图 8 直接比较两平台的延迟曲线。鲲鹏 PBS 上 IVF-PQ 从 $np=1$ 的 $571.67\ \mu s$ 降到 $np=4$ 的 $311.45\ \mu s$ ，随后 $np=8$ 回升到 $532.36\ \mu s$ ；Block-HNSW 从 $np=1$ 的 $341.73\ \mu s$ 降到 $np=4$ 的 $248.00\ \mu s$ ，但 $np=8$ 上升到 $776.99\ \mu s$ 。这个拐点说明，在当前数据量和 query batch 下，增加进程数并不总是等价于更快：当每个 rank 的本地任务过小，固定通信、合并、调度、HNSW 图构建差异和节点负载噪声会更突出。

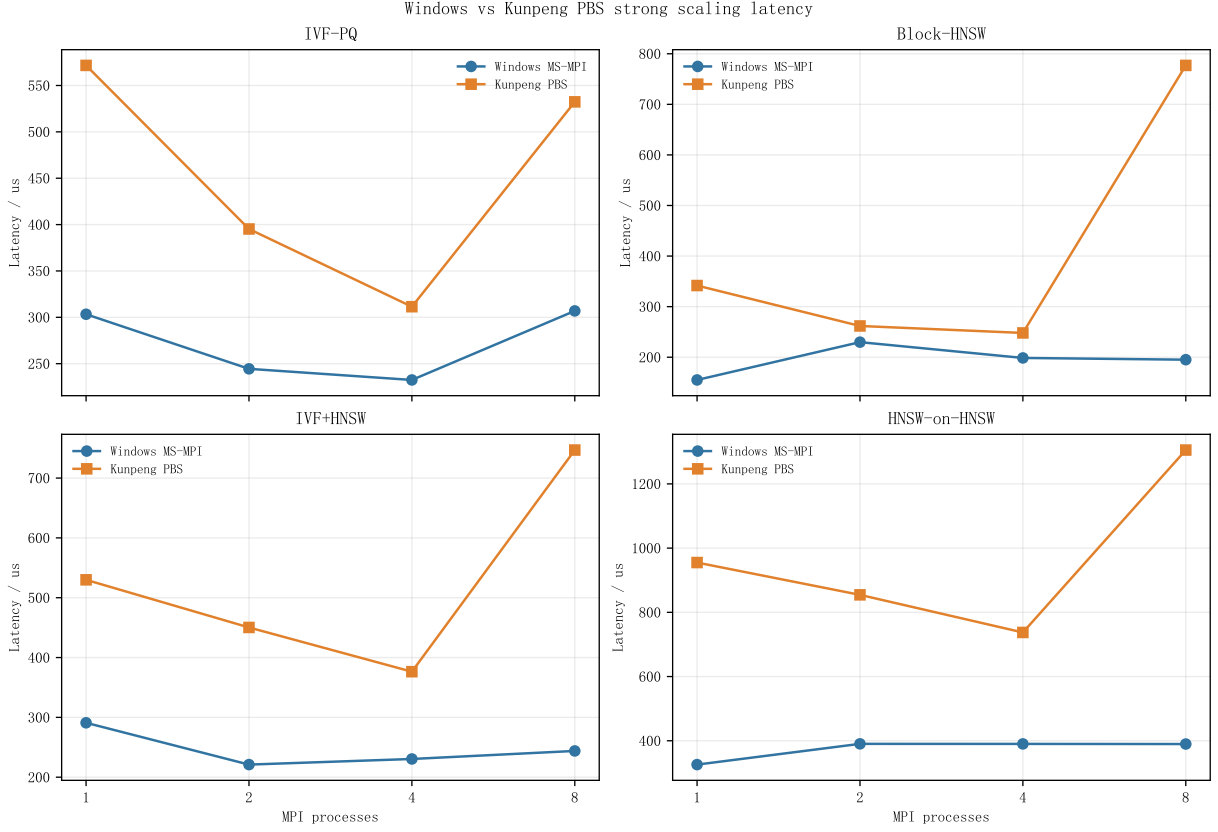


图 8 Windows MS-MPI 与鲲鹏 PBS 的强扩展延迟曲线。

6.3 扩展性数据表

表 7 汇总两平台每个算法的最好进程数。从结果看, IVF-PQ 在两平台都在 4 个进程附近最好; Block-HNSW 在鲲鹏 PBS 的 4 个进程最好, 但 Windows 上单进程更快; IVF+HNSW 在 Windows 上 2 个进程最好, 在鲲鹏上 4 个进程最好; HNSW-on-HNSW 在 Windows 上单进程最好, 在鲲鹏上 4 个进程最好。

平台	算法	最佳 np	最低延迟	np=8 延迟	np=8 效率
Windows	IVF-PQ	4	232.45	306.91	0.12
Windows	Block-HNSW	1	155.03	195.11	0.10
Windows	IVF+HNSW	2	221.06	243.89	0.15
Windows	HNSW-on-HNSW	1	325.77	389.81	0.10
鲲鹏 PBS	IVF-PQ	4	311.45	532.36	0.13
鲲鹏 PBS	Block-HNSW	4	248.00	776.99	0.05
鲲鹏 PBS	IVF+HNSW	4	376.36	746.84	0.09
鲲鹏 PBS	HNSW-on-HNSW	4	737.31	1305.40	0.09

表 7 强扩展实验摘要。延迟单位为 $\mu\text{s}/\text{query}$, 效率以 $\text{np}=1$ 为基准。

6.4 Amdahl 视角

用 Amdahl 定律估计并行效率时, 可将不可并行部分粗略视为通信、merge、同步、调度和无法随 shard 缩小线性下降的图遍历常数项。设 $\text{np}=4$ 的速度提升为 S_4 , 则串行比例可估为

$$\alpha \approx \frac{1/S_4 - 1/4}{1 - 1/4}.$$

鲲鹏 PBS 上 IVF-PQ 的 $S_4 = 571.67/311.45 = 1.84$, 得到 $\alpha \approx 0.39$; Windows IVF-PQ 的 $S_4 = 303.31/232.45 = 1.30$, 得到 $\alpha \approx 0.69$ 。这个估计不是严格算法串行比例, 而是把小规模 shard 下的

额外成本也折算进“不可扩展部分”。它说明在 100K base、2000 query 的规模下，MPI 强扩展已经不是理想均匀计算模型，通信和运行时常数需要与本地搜索一起考虑。

6.5 MPI/OpenMP 混合布局

前面的强扩展实验固定 `threads=2`，只改变 MPI 进程数。为了更接近实际调参过程，本节在 Windows MS-MPI 上进一步固定总 worker 预算，比较 $np \times threads$ 的不同分配方式。实验覆盖 8 个 worker 下的 1×8 、 2×4 、 4×2 、 8×1 ，以及 16 个 worker 下的 1×16 、 2×8 、 4×4 、 8×2 。HNSW-on-HNSW 在本节固定 `nprobe_blocks=4`，用于观察布局对调度和访存的影响；高召回配置仍以前文 `nprobe_blocks=16` 的参数扫描结果为准。

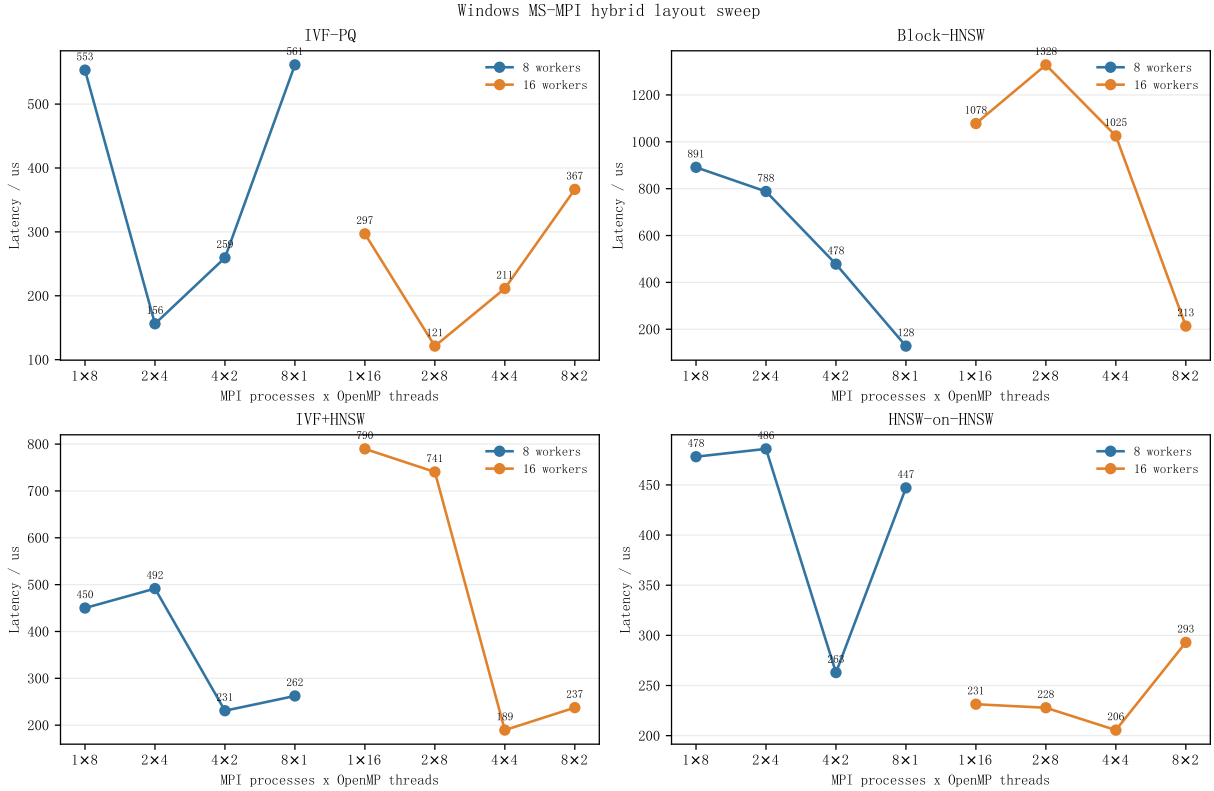


图 9 Windows MS-MPI 上固定 worker 预算的 $np \times threads$ 布局实验。

图 9 显示，不同算法适合的布局并不相同。IVF-PQ 在 16 worker 预算下以 2×8 最快，延迟为 $121.23 \mu s/query$ ；它既保留了 MPI shard 缩小带来的扫描减少，也让每个 rank 内仍有足够线程处理倒排表。Block-HNSW 更偏向更多 MPI shard，8 worker 下 8×1 达到 $128.05 \mu s/query$ ，说明图搜索中 rank 内并行收益有限，缩小本地图规模更直接。IVF+HNSW 的最好点是 4×4 ，延迟为 $189.48 \mu s/query$ ，介于倒排筛选和图遍历之间。HNSW-on-HNSW 在该中等探测配置下也是 4×4 最稳，过多 rank 会增加 top-level block 选择和候选合并的尾部。

6.6 Windows 亲和性实验

本地 Windows 平台是 P-core/E-core 混合架构。为了检查运行时调度对 MPI 作业的影响，本节使用同一个 MPI 二进制、同一组 `np=4, threads=2, query_n=2000` 参数，通过 `cmd /AFFINITY` 分别测试默认调度、仅 P-core、仅 E-core，以及 P/E 全开。该实验仍然是真实 MS-MPI 运行，只改变进程可见的逻辑核集合。

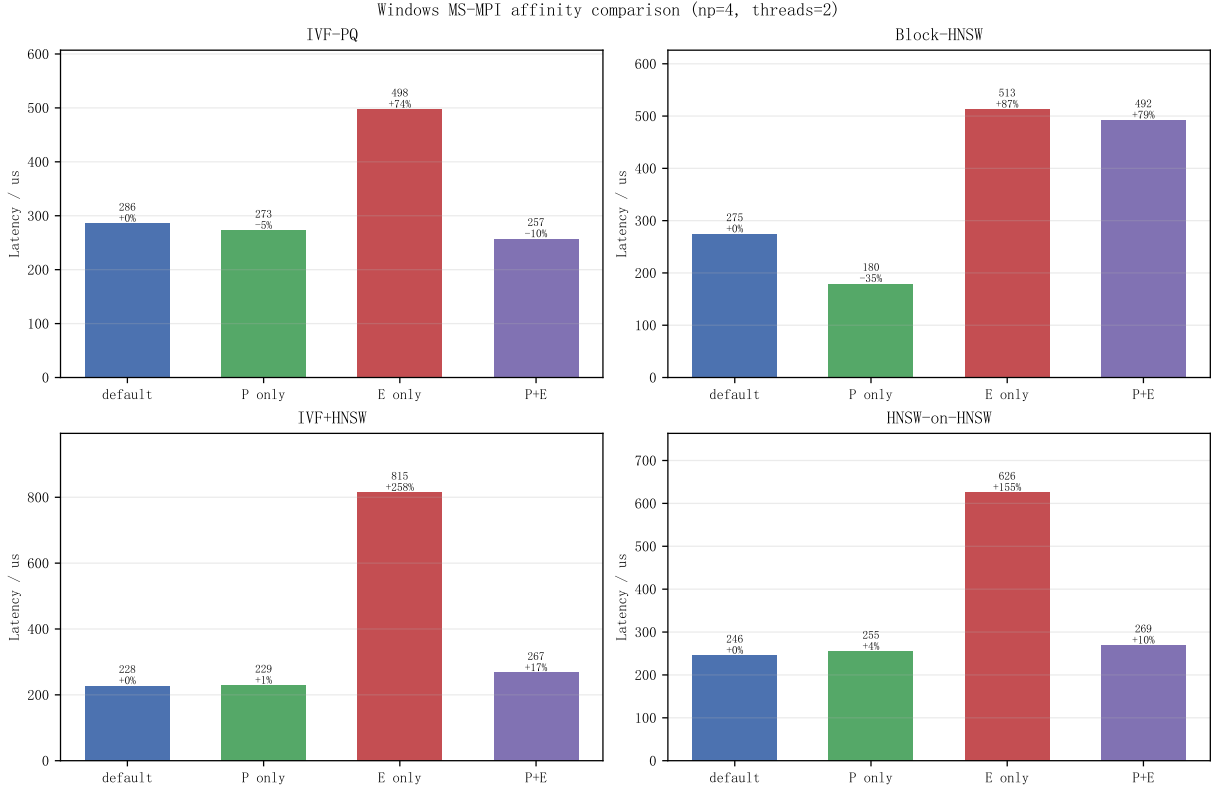


图 10 Windows MS-MPI 亲和性实验，固定 np=4, threads=2。

结果中 E-core-only 对所有算法都明显变慢，IVF+HNSW 从默认的 227.71 $\mu\text{s}/\text{query}$ 增加到 814.81 $\mu\text{s}/\text{query}$ ，HNSW-on-HNSW 也增加到 625.63 $\mu\text{s}/\text{query}$ 。这与后文 VTune uarch 中大量采样落在 E-core、Back-End Bound 和 Memory Bound 较高的现象一致。P-core-only 对 Block-HNSW 最有帮助，延迟从 274.61 降到 179.60 $\mu\text{s}/\text{query}$ ；IVF-PQ 则在 P/E 全开时最快，说明规则扫描和多 rank 并发能利用更多逻辑核。实际复现实验时，我保留默认调度作为跨平台主结果，同时把亲和性作为解释本地波动和混合架构影响的补充证据。

7 通信开销与负载均衡

7.1 通信与 merge 随进程数变化

图 11 左侧展示通信与 merge 开销随进程数的变化。两平台日志都显示 comm+merge latency 从 np=1 到 np=8 增大，但绝对值只有微秒级。以鲲鹏 PBS 为例，np=8 下 IVF-PQ 的通信/merge 为 2.88 μs ，Block-HNSW 为 2.55 μs ，IVF+HNSW 为 3.37 μs ，HNSW-on-HNSW 为 3.05 μs 。与数百到上千微秒的本地搜索相比，这部分还不是主瓶颈。

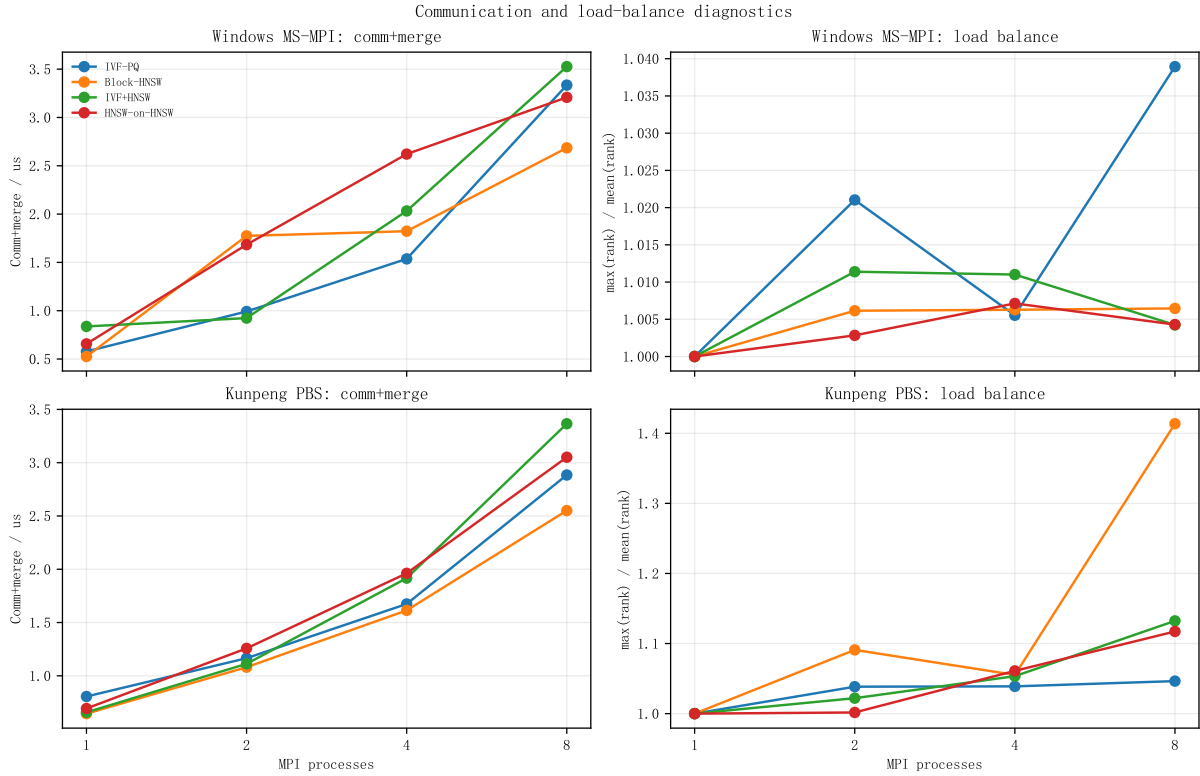


图 11 通信/merge 开销与 rank 负载不均衡度。

7.2 通信占比

表 8 使用代表参数点计算通信与 merge 占总延迟比例。Windows 上 HNSW-on-HNSW 因总搜索时间很长，通信占比仅 0.37%；鲲鹏 PBS 上四种算法的通信占比均不超过 0.53%。这说明本实验规模中，优化方向应优先放在本地索引结构和 rank 内搜索，而不是立即重写通信拓扑。

平台	IVF-PQ	Block-HNSW	IVF+HNSW	HNSW-on-HNSW
Windows MS-MPI	1.04%	1.20%	1.11%	0.37%
鲲鹏 PBS	0.53%	0.34%	0.33%	0.12%

表 8 代表参数点中通信与 merge 占平均总延迟比例

7.3 负载不均衡分析

负载不均衡度采用 $\max(T_i)/\text{mean}(T_i)$ 。Windows 上代表参数点均低于 1.07，说明本地机器的各 rank 搜索时间比较接近。鲲鹏 PBS 上图索引的不均衡更明显：代表参数点中 Block-HNSW 为 1.19441，HNSW-on-HNSW 为 1.17062；扩展性实验 np=8 时 Block-HNSW 甚至达到 1.41。这说明图搜索路径与数据分布有关，即使 base 数量均分，不同 rank 的图遍历代价仍可能不同。

从算法结构看，IVF-PQ 的扫描量由 selected list 长度决定，list 长度不均会引入尾部；Block-HNSW 的搜索路径由图邻接和 query 位置决定，某些 shard 可能需要扩展更多节点；IVF+HNSW 同时受聚类 and 图搜索影响；HNSW-on-HNSW 又加入了 top-level block 选择，若探测全部 block，负载更接近 block 内图搜索总和。

7.4 改进方向

如果后续要进一步优化负载均衡，可以考虑三种做法。第一，将 base 在构建前随机打散，使连续区间 shard 更接近全局分布；第二，对 IVF inverted list 按 list 粒度重新分配，而不是按 base id 连续切分；第三，对图索引记录每个 rank 的历史搜索时间，将慢 rank 的 block 拆得更细。当前实

现选择连续切分，是因为它简单、可复现、通信位移清楚，并且在代表参数下已经能保持可接受的不均衡。

8 VTune 性能剖析

8.1 Hotspots 采集口径

前文的参数扫描、扩展性、混合布局和亲和性实验均使用真实 MPI 二进制。VTune 部分的目的不同：它不再重复端到端 MPI 延迟，而是解释每个 rank 内部的本地 HNSW 搜索为什么成为主要瓶颈。Windows 上直接把整个 MS-MPI 作业交给 VTune 时，采样容易落在 launcher 和等待路径上；本次也保留了 `run_vtune_mpi_hotspots.ps1` 作为 rank 0 包装尝试，但在本机 MS-MPI 下该路径会在子进程包装阶段挂住，未作为正文数据来源。

因此本节使用与 MPI rank 内相同的搜索内核，对 `ANN_NO_MPI profiling` 二进制进行符号化 Hotspots 和 uarch 采集。这样可以稳定把地址映射到 HNSW 搜索、AVX 内积和候选队列函数；其结论用于解释“本地搜索为什么比通信更贵”，不替代前文的真实 MPI timing。为了让 VTune 能把地址映射回函数，本节重新编译了带调试符号的 profiling 版本：

```
1 g++ main.cc -o build/main_no_mpi_profile.exe -O2 -g -std=c++11 -I. \
2   -fopenmp -lpthread -mavx2 -mfma -DANN_NO_MPI
3 vtune -collect hotspots -result-dir results/vtune_hotspots_hnsw_profile \
4   -- build/main_no_mpi_profile.exe 2 16 50 1000 2000 hnsw
```

目标 workload 固定为 HNSW: `threads=2`、`m=16`、`ef=50`、`query_n=2000`。运行时输出 `Recall@10=0.99985`，平均延迟为 `178.50710  $\mu$ s/query`。采集结果见表 9。

指标	数值	说明
Elapsed Time	20.450 s	端到端采样时间。
CPU Time	19.922 s	所有线程 CPU 时间合计。
Effective Time	16.713 s	有效执行时间。
Spin Time	3.209 s	等待或自旋时间，约占 CPU Time 的 16.1%。
Total Thread Count	2	与运行参数 <code>threads=2</code> 一致。

表 9 VTune Hotspots 汇总，来源于符号化 profiling 的 summary 导出

8.2 热点函数

符号化后的前五个热点见表 10。其中距离计算相关的 AVX intrinsic wrapper 与 AVX 内积函数合计占据显著比例，说明向量内积仍是局部 HNSW 搜索中的高频路径；`searchBaseLayer`、优先队列操作和 `pthread_mutex_lock` 则说明图遍历、候选队列维护、OpenMP/pthread runtime 同步也贡献了明显开销。

Function	Module	CPU Time	Share
<code>_mm256_mul_ps wrapper</code>	profile exe	3.218 s	16.2%
<code>pthread_mutex_lock</code>	pthread dll	2.664 s	13.4%
<code>searchBaseLayer</code>	profile exe	2.493 s	12.5%
<code>InnerProduct AVX</code>	profile exe	2.098 s	10.5%
<code>_mm256_add_ps wrapper</code>	profile exe	1.663 s	8.3%

表 10 符号化 VTune Hotspots 前五项

8.3 汇编观察

为了把 Hotspots 与实际指令对应起来，本节使用 `objdump -Cd -Mintel` 导出了关键函数汇编，摘录文件为 `assembly_excerpt.txt`。首先看 AVX 内积函数的核心循环：

```
vmovups ymm1, YMMWORD PTR [rdx]
vmulps ymm1, ymm1, YMMWORD PTR [rcx]
add rcx, 0x40
add rdx, 0x40
vmovups ymm2, YMMWORD PTR [rdx-0x20]
vaddps ymm0, ymm1, ymm0
vmovups ymm1, YMMWORD PTR [rcx-0x20]
vmulps ymm1, ymm1, ymm2
vaddps ymm0, ymm1, ymm0
cmp rcx, rax
jb <InnerProductSIMD16ExtAVX+0x30>
vextractf128 xmm0, ymm0, 0x1
vshufps xmm2, xmm0, xmm0, 0x55
vaddss xmm1, xmm1, xmm0
vzeroupper
```

该循环每轮步进 $0x40=64$ 字节，对应 16 个 float。DEEP100K 的向量维度为 96，因此一次距离计算正好经历 6 个 16-float AVX 块。内循环包含 `vmovups`、`vmulps`、`vaddps` 和最后的水平归约，说明距离内核确实已经走 AVX 向量化路径；BuildIndex 中也能看到 `AVXCapable()` 后把函数指针设置为 `InnerProductSIMD16ExtAVX` 和 `InnerProductDistanceSIMD16ExtAVX`。

另一方面，`hnsu_search_multi_entry_omp` 和 `searchBaseLayer` 的汇编片段包含大量 `cmp/jcc` 分支、队列插入/弹出、visited 标记判断和异常清理路径。它不像内积 kernel 那样是规则线性扫描，而是随 query 和图邻居关系跳转。结合表 10 中 `searchBaseLayer`、priority queue 的 `pop/emplace`、比较器等函数占比，可以判断瓶颈不是“没有 SIMD”，而是 SIMD 距离计算嵌在不规则图遍历和候选队列维护之中。

8.4 剖析解释

Hotspots 与前文结果相互印证。HNSW 类方法达到高召回，但延迟不一定最低，原因在于图搜索需要维护候选队列、访问 visited 标记并沿邻居边扩展；这些操作难以像 Flat 或 PQ 扫描那样形成规则 SIMD 流水线。对 MPI 版本而言，通信只有微秒级，而一次 HNSW 本地搜索可达到数百到数千微秒，因此 rank 内图遍历和候选管理才是优化重点。

8.5 uarch-exploration 与硬件事件采集

本次先在普通 Windows 会话尝试运行 `uarch-exploration`。失败日志位于 VTune 导出目录，对应文件为：

```
1 uarch_collect_log.txt
```

该路径返回如下诊断：

```
1 Cannot enable Hardware Event-Based Sampling or Hardware Tracing.
2 Run the product as administrator to collect hardware events.
```

随后使用管理员脚本重新采集同一 no-MPI workload。采样驱动检查显示管理员权限与三个采样服务均可用：

```
1 User has admin rights: OK
2 sepdrv5 service is running
3 sepdal service is running
4 vtss service...OK
```

管理员采集成功导出 summary、top-down、hardware events 和 hotspots 四类报告，表 11 给出正文使用的关键指标。

指标	实测值	解释
Elapsed Time	49.970 s	管理员 <code>uarch-exploration</code> 采集耗时。
Clockticks / Instructions	12.106B /	采样窗口内的周期与退休指令数。
Retired	8.242B	
CPI Rate	1.469	CPI 偏高, 说明存在内存停顿、分支或长延迟操作。
MUX Reliability	0.952	多路复用可靠性较高, 硬件事件可用于趋势判断。
E-core Retiring	19.7%	有效退休槽位占比较低。
E-core Front-End Bound	9.0%	前端不是主要瓶颈。
E-core Bad Speculation	11.2%	图遍历分支和队列比较造成一定错误推测。
E-core Back-End Bound	60.1%	主要瓶颈在后端。
E-core Memory Bound	50.4%	后端停顿主要来自内存层次。
E-core L1 Bound / Load	12.1% / 11.2%	visited 标记、邻接表和向量地址访问带来 TLB 压力。
STLB Miss		
E-core DRAM Bound	30.1%	不规则图访问会落到更远内存层级。

表 11 管理员权限下 VTune uarch-exploration 关键指标, 来源于 `uarch_admin_summary.txt`

需要说明的是, 本地机器是 P-core/E-core 混合架构, 本次 workload 的绝大多数 clockticks 与 retired instructions 落在 E-core 上: E-core clockticks 为 11.929B, 而 P-core 仅 0.177B。因此表 11 主要引用 E-core Top-Down 指标, P-core 子树中部分 0.0% 或 FPU capacity 指标不作为全局结论。函数级 `uarch_admin_hotspots.csv` 也支持这一判断: `searchBaseLayer` 的 E-core Memory Bound 为 47.1%, Load STLB Miss 为 35.8%, Bad Speculation 为 22.1%; `InnerProductSIMD16ExtAVX` 和 `_mm256_mul_ps` 仍是热点, 但它们嵌在图遍历和候选队列维护之中。

因此, 硬件事件与汇编观察给出的结论一致: 距离计算路径已经使用 AVX 指令, SIMD 并不是缺失项; 更主要的限制来自 HNSW 搜索的随机邻接访问、visited/TLB 压力、候选优先队列比较以及少量同步开销。对于 MPI 版本, 通信/merge 只占微秒级, 优化优先级仍应放在 rank 内图访问局部性和候选维护上。

9 阻塞与非阻塞通信实验

9.1 实现方式

默认程序使用阻塞 `MPI_Bcast` 与 `gather`。为了比较通信模式, `gather` 包装处增加了一个环境变量开关:

```
1 USE_NONBLOCKING_MPI=1
```

阻塞模式走常规 `gather`; 非阻塞模式使用 `MPI_Igather` 并在数据真正需要前 `MPI_Wait`。由于本程序的本地搜索必须在 `gather` 前完成, 非阻塞 `gather` 主要减少 runtime 等待和进程到达时间差的影响, 并不能完全重叠本地搜索。

9.2 鲲鹏 PBS 结果

图 12 和表 12 给出鲲鹏 PBS 的通信模式实验。四种算法的 Recall 在阻塞与非阻塞模式下完全一致。非阻塞模式在 IVF-PQ 上降低 5.13%, 在 IVF+HNSW 上降低 6.37%, 在 Block-HNSW 上降低 0.52%; HNSW-on-HNSW 则慢 0.89%。这说明非阻塞通信在当前实现中不是稳定万能优化, 而是取决于本地搜索结束时间差、runtime 调度和 `gather` 数据到达节奏。

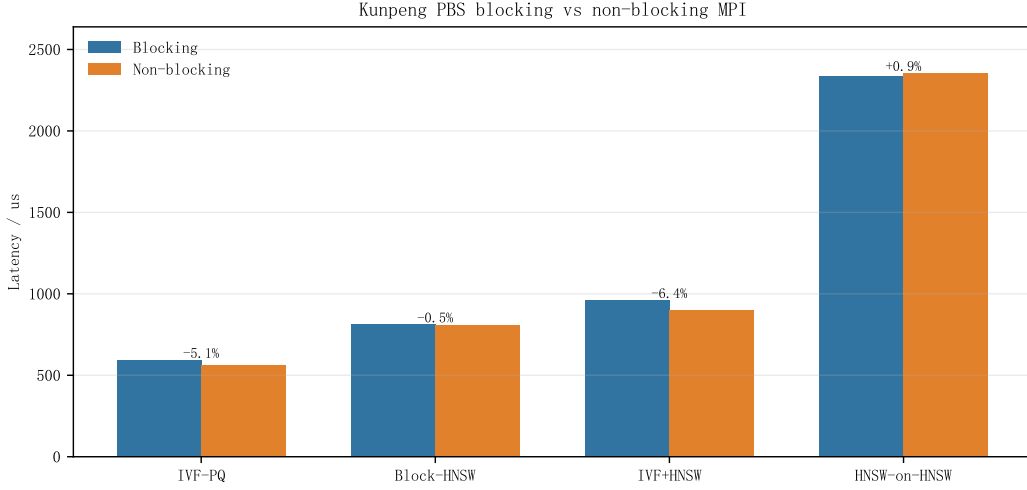


图 12 鲲鹏 PBS 上阻塞与非阻塞 MPI 的延迟对比。

算法	Blocking	Nonblocking	变化	Recall
IVF-PQ	594.79821	564.27193	-5.13%	0.96515
Block-HNSW	812.47854	808.22515	-0.52%	1.00000
IVF+HNSW	961.82323	900.58351	-6.37%	0.96450
HNSW-on-HNSW	2335.40905	2356.10485	+0.89%	0.99995

表 12 鲲鹏 PBS 阻塞/非阻塞通信模式对比。延迟单位为 $\mu\text{s}/\text{query}$ 。

9.3 通信模式结论

从数据上看，非阻塞 MPI 的正确性没有问题，四种算法 Recall 完全一致；性能上有小幅改善也有轻微退化。根本原因是本实验的通信数据量很小，且 gather 发生在本地搜索之后，真正可以重叠的空间有限。若要让非阻塞通信产生更稳定收益，需要把 query batch 切成多个 tile：当 tile i 的候选在后台 gather 时，rank 可以搜索 tile $i + 1$ 。这会改变代码结构和结果缓冲方式，因此更适合作为后续版本的通信流水线优化。

10 结果可信性与可复现性检查

10.1 正确性一致性

四种模式在 Windows MS-MPI 与鲲鹏 PBS 中的代表参数 Recall 完全一致，尤其是 IVF-PQ 的 0.96515、IVF+HNSW 的 0.96450、Block-HNSW 的 1.00000 以及 HNSW-on-HNSW 的 0.99995 在两平台中一致。这说明跨 rank 候选 id 偏移、gather 缓冲布局 and rank 0 top- k merge 没有平台相关错误。

10.2 计时一致性

所有日志中都有

$$\text{latency} \approx \text{max local search} + \text{comm} + \text{merge}.$$

例如 Windows IVF-PQ 中 $331.10550 + 3.49005 = 334.59555$ ，与总延迟完全对齐；鲲鹏 PBS 的 HNSW-on-HNSW 中 $2486.21905 + 2.94542 = 2489.16447$ ，与总延迟完全对齐。这说明 $\text{comm} + \text{merge latency}$ 的计算口径稳定，不是事后人为估计。

10.3 脚本可复现

本地绘图脚本已固定为：

```
1 micromamba run -n test python scripts/parse_mpi_results.py
```

脚本会读取四个双平台原始日志以及两个 Windows 进阶 CSV，生成结构化 CSV 和九张 PDF 图。每张图都位于 `report/fig/`，并由本报告直接引用。服务器自动脚本读取 `KUNPENG_PASSWORD` 环境变量，不在仓库中写入密码；PBS 作业脚本通过 `NP`、`OMP_NUM_THREADS`、`QUERY_N`、`NPROBE`、`HNSW_EF` 等变量控制实验参数，复现实验不需要手改源代码。

11 问题与改进方向

更多重复次数。本次已经完成全量参数扫描和扩展性实验，但每个点主要记录一次全量运行。若作为论文级 benchmark，应在服务器低负载时重复 5 次以上取中位数和四分位区间。本课程实验更关注算法和 MPI 数据流，当前日志已足以证明正确性和主要趋势。

HNSW-on-HNSW 参数。`nprobe_blocks=16` 是偏向高召回的配置，相当于在 top-level graph 之后仍搜索全部 block。本次已经扫描 1,2,4,8,16 并画出 Recall@10 与延迟曲线；若把它作为低延迟方案，还需要在 4 到 16 之间补更细粒度的探测点，寻找召回刚接近目标值时的拐点。

更细的负载均衡。当前 base 按连续区间切分，简单稳定，但没有根据 inverted list 长度、HNSW 图度数或查询热度做动态均衡。如果某些 rank 的局部图搜索持续更慢，可以考虑按随机打散、质心均衡或 list 粒度分配来减小尾部延迟。

通信优化。当前所有候选都 gather 到 rank 0。对于更多 rank，可以改为分层合并：先在节点内合并，再跨节点合并；或者使用自定义 top-*k* 归约。但在本实验记录中通信/merge 占比很低，因此这不是当前首要瓶颈。

Profiling 深度。本次 VTune Hotspots 已用带符号二进制映射到 HNSW 搜索、AVX 内积和优先队列路径，汇编也验证了 16-float AVX 距离内核；管理员权限下的 uarch 采集进一步给出了 Back-End Bound、Memory Bound、Bad Speculation、L1/TLB 与 DRAM 指标。若继续深化，可以再做 Memory Access 分析，把具体内存对象和源码行绑定起来。

12 结论

本实验完成了 ANN 搜索从单机多线程到 MPI 多进程环境的迁移，并保留 OpenMP 作为 rank 内并行层。IVF-PQ 验证了基础倒排索引迁移；Block-HNSW、IVF+HNSW 和 HNSW-on-HNSW 分别代表三种图索引组织方式。双平台参数扫描显示，相同参数下 Recall 在 Windows 与鲲鹏 PBS 上完全一致，说明分布式 top-*k* 合并逻辑正确；通信与 merge 开销在当前验证规模中很小，主要瓶颈仍是本地 ANN 搜索。

从算法效果看，IVF-PQ 与 IVF+HNSW 的召回略低但保持较好的筛选效率；Block-HNSW 在代表参数下达到 Recall@10=1.00000，是当前最可靠的高召回图索引方案；HNSW-on-HNSW 证明了分层图结构的可行性和高召回能力，但全 block 探测配置带来了明显延迟代价。强扩展实验表明，`np=4` 通常比 `np=8` 更稳，说明在 DEEP100K 和 2000 query 的规模下，MPI 强扩展存在固定开销与负载不均衡拐点。VTune、硬件事件与汇编分析进一步说明，距离内核已经使用 AVX SIMD，而 E-core Back-End Bound 和 Memory Bound 分别达到 60.1% 与 50.4%，后续优化更应关注图遍历、候选队列、TLB/DRAM 压力和负载尾部。

A 复现命令与结果文件

A.1 本地复现

```

1 cd D:\Study\26sp\parallel\ann-mpi
2 make local
3 powershell -ExecutionPolicy Bypass -File scripts/run_parameter_sweep.ps1
4 powershell -ExecutionPolicy Bypass -File scripts/run_scalability.ps1
5 powershell -ExecutionPolicy Bypass -File scripts/run_hybrid_layout.ps1
6 powershell -ExecutionPolicy Bypass -File scripts/run_affinity.ps1
7 micromamba run -n test python scripts/parse_mpi_results.py

```

Windows VTune profiling 的普通 Hotspots 和管理员 uarch 采集入口如下：

```

1 g++ main.cc -o build/main_no_mpi_profile.exe -O2 -g -std=c++11 -I. \
2   -fopenmp -lpthread -mavx2 -mfma -DANN_NO_MPI
3 powershell -ExecutionPolicy Bypass -File scripts/run_vtune_uarch_admin.ps1

```

也可以右键管理员脚本运行；本次已导出 uarch_admin_summary.txt、uarch_admin_top_down.txt、uarch_admin_hw_events.csv 与 uarch_admin_hotspots.csv。

A.2 鲲鹏 PBS 复现

```

1 cd ann-mpi
2 make clean
3 make
4 qsub -v NP=8,OMP_NUM_THREADS=2,QUERY_N=2000 qsub_mpi.sh

```

完整批量实验可使用自动脚本提交和回收结果：

```

1 set KUNPENG_PASSWORD=<server_password>
2 micromamba run -n test python scripts/run_kunpeng_pbs_sweep_auto.py
3 micromamba run -n test python scripts/run_kunpeng_pbs_comm_auto.py

```

A.3 生成产物

文件	说明
parameter_sweep 20260525_141037.csv	Windows 参数扫描结构化结果，24 行。
scalability 20260525_145526.csv	Windows 强扩展结构化结果，16 行。
hybrid_layout 20260525_205224.csv	Windows np × threads 混合布局结果，32 行。
affinity 20260525_205832.csv	Windows P-core/E-core 亲和性结果，16 行。
kunpeng_pbs_sweep 20260525_153105.csv	鲲鹏 PBS 参数扫描与扩展性结构化结果，40 行。
kunpeng_pbs_comm_modes 20260525_154940.csv	鲲鹏 PBS 阻塞/非阻塞通信模式结果，8 行。
results/vtune_exports/ report/fig/*.pdf	VTune Hotspots、汇编摘录、uarch 普通会话日志和管理员实测导出位置。 由解析脚本从 CSV 生成的所有图。

表 13 报告引用的结构化结果和图表产物

B 实验矩阵明细

B.1 参数扫描矩阵

表 14 列出本次双平台参数扫描的完整覆盖范围。Windows 与鲲鹏 PBS 使用相同参数集合，因此正文图 5 可以直接比较两平台曲线，而不是把不同参数点混在一起解释。

算法	sweep 参数	固定参数
IVF-PQ	nprobe=1,2,4,8,16	nlist=16,p=1000,np=8,threads=2,query_n=2000
IVF-PQ	nlist=8,16,32,64	nprobe=4,p=1000,np=8,threads=2,query_n=2000
Block-HNSW	ef=10,20,50,100,200	m=16,np=8,threads=2,query_n=2000
IVF+HNSW	nprobe=1,2,4,8,16	nlist=16,ef=50,np=8,threads=2,query_n=2000
HNSW-on-HNSW	nprobe_blocks =1,2,4,8,16	nblocks=16,ef=50,np=8,threads=2,query_n=2000

表 14 参数扫描实验矩阵

B.2 扩展性矩阵

扩展性实验统一固定每个算法的代表参数，只改变 MPI 进程数。表 15 中的设置保证了强扩展曲线反映“同一算法同一 recall 目标下的进程数变化”，而不是参数变化带来的混合影响。

算法	进程数	固定参数
IVF-PQ	np=1,2,4,8	threads=2,nlist=16,nprobe=4,p=1000,query_n=2000
Block-HNSW	np=1,2,4,8	threads=2,m=16,ef=50,query_n=2000
IVF+HNSW	np=1,2,4,8	threads=2,nlist=16,nprobe=4,ef=50,query_n=2000
HNSW-on-HNSW	np=1,2,4,8	threads=2,nblocks=16,nprobe_blocks=4 ef=50,query_n=2000

表 15 强扩展实验矩阵

B.3 复现注意事项

复现实验时有三点需要保持一致。第一，Windows 与鲲鹏 PBS 都应使用同一份 DEEP100K 数据，避免不同数据目录造成 recall 差异。第二，绘图脚本必须在 micromamba 的 test 环境中运行，因为该环境已经包含 matplotlib、numpy 和服务自动化所需的 paramiko。第三，PBS 实验以作业输出为准，正文不展示非 PBS 路径的服务器数据；这样与提交指南的队列运行方式保持一致。

参考文献

- [1] MPI Forum. *MPI: A Message-Passing Interface Standard*. Version 4.1, 2023.
- [2] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 2011.
- [3] Y. Malkov and D. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE TPAMI*, 2020.
- [4] A. Babenko and V. Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. *CVPR*, 2016.